ADSC 4050 Project

S&P500 Clustering and Volatility Forecast

Date: Friday 17, 2026

Prepared by:

1. Mint Thai - T00762325
2. Cynthia Urrutia - T00735328

Introduction

Forecasting how much a stock price will fluctuate is one of the most important challenges in finance. It drives everything from portfolio risk management to options pricing to investment decisions. However, stock markets do not move in a uniform manner. Some stocks are stable, steady growers, while others swing wildly with earnings surprises, macro events, or shifts in investor sentiment. This raises a natural question: how many distinct groups of stocks exist in the S&P 500, and can grouping them improve the accuracy of volatility prediction?

This project aims to identify natural stock groupings within the S&P 500 and evaluate whether incorporating that structure improves volatility forecasting. K-means clustering is applied using features derived from returns, rolling volatility, and trading activity. The resulting cluster labels are then incorporated into a Long Short-Term Memory (LSTM) model trained to forecast 21-day forward volatility.

The dataset covers five years of daily price data (open, high, low, close) and trading volume across 503 S&P 500 companies. Sector representation within the index is unequal; technology and financials make up a larger share, while energy and utilities contribute less, reflecting differences in market composition.

The dataset is split into training (March 2021 - March 2024), validation (March 2024 - March 2025), and test (March 2025 - March 2026) periods to prevent data leakage. Only stocks present across all three periods are retained, resulting in 499 stocks across 3 subsets. Feature construction differs between the two tasks. For clustering, features summarize stock behavior through returns, rolling volatility, and volume changes over 1-day, 5-day, and 21-day horizons. For forecasting, features include daily absolute returns, rolling standard deviations over 5, 10, and 21 days, intraday price range averages over 5 and 21 days, relative trading volume, and cluster-based indicators.

Research Questions

The two main research questions are:

1. RQ1: What distinct behavioral groups exist among S&P 500 stocks, and can K-means clustering reliably separate stocks by their return, volatility, and trading activity profiles?

2. RQ2: Does incorporating cluster membership as a feature improve LSTM-based 21-day volatility forecasting compared to a naive baseline?

To answer these, K-means clustering is first applied to identify groups of stocks with similar statistical behavior. The resulting cluster labels are then incorporated into an LSTM model trained to forecast future volatility. A naive baseline, using the most recently observed 21-day volatility as the forecast, serves as the performance benchmark. Model accuracy is evaluated using RMSE, MAE, and R^2.

Model Construction

This section describes the construction of the clustering and forecasting models, including feature engineering, mathematical definitions of variables, and the rationale behind the selected methodologies.

K-means Clustering

K-means clustering is selected for its simplicity and interpretability. Compared to hierarchical clustering, K-means produces more stable and consistent groupings across the training, validation, and test periods. The optimal number of clusters is determined using the silhouette score and elbow method.

Clustering Feature Construction

Features are derived from closing prices and trading volume. Let $P_{i,t}$ denote the closing price of stock i at time t, and $V_{i,t}$ its trading volume. Feature construction focuses on three key dimensions of stock behavior, as described below:

- **Returns:** To stabilize variance and improve comparability across stocks, prices are transformed using the logarithmic function, defined as $\log\backslash_\text{close}_{i,t} = \log(P_{i,t})$. Log returns over different horizons $k \in \{1, 5, 21\}$ are then computed as $r_{i,t}^{(k)} = \log(P_{i,t}) - \log(P_{i,t-k})$.

- **Risk (Volatility):** Volatility is measured as the rolling standard deviation of daily log returns, defined as $\sigma_{i,t}^{(k)} = \mathrm{sd}(r_{i,t-k+1}^{(1)}, \ldots, r_{i,t}^{(1)})$, where $\mathrm{sd}(\cdot)$ denotes the sample standard deviation.

- **Trading Activity:** To reduce skewness in trading activity, volume is transformed using $\log_\mathrm{volume}_{i,t} = \log(\max(V_{i,t}, 1))$. Changes in trading activity are then computed as $\Delta v_{i,t}^{(k)} = \log_\mathrm{volume}_{i,t} - \log_\mathrm{volume}_{i,t-k}$.

Feature Cases

Three feature configurations are tested using $k = 1, 5, 21$, capturing different temporal dynamics. These correspond to immediate (1-day), short-term (5-day), and medium-term (21-day) horizons, allowing the clustering model to distinguish between short-lived fluctuations and more persistent patterns in stock behavior.

Since clustering is performed at the stock level, features are aggregated over time. For each variable $x_{i,t}$, the mean and standard deviation are computed as:

- Mean: $\bar{x}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} x_{i,t}$
- Standard deviation: $s_i(x) = \sqrt{\frac{1}{T_i - 1} \sum_{t=1}^{T_i} (x_{i,t} - \bar{x}_i)^2}$

Volatility Forecasting Model

A Long Short-Term Memory (LSTM) neural network is selected for the forecasting task due to its ability to model complex, nonlinear relationships and retain long-term dependencies in sequential data. Volatility is not a static measure because it evolves over time, and recent market conditions carry information about future uncertainty, making LSTMs a natural fit compared to simpler regression models that require explicit lag specification. Other forecasting approaches, including Random Forest, were explored but yielded lower predictive performance and therefore were not selected in this analysis.

Target Variable

Let $C_{i,t}$ denote the closing price of stock i at time t. Daily returns are first computed as: $R_{i,t} = \frac{C_{i,t} - C_{i,t-1}}{C_{i,t-1}}$, capturing the relative price change from one day to the next.

The target is the realized volatility over the next 21 trading days: $Y_{i,t} = \mathrm{sd}(R_{i,t+1}, R_{i,t+2}, \ldots, R_{i,t+21})$, representing the level of market uncertainty the model aims to predict.

Feature Construction

Nine features are constructed to capture different dimensions of recent market behavior. At each prediction point, each feature produces one value per day. The LSTM then receives the last 21 consecutive daily observations of all 9 features as a sequence, allowing the model to learn how each feature has been evolving over time, not just its most recent value.

Feature construction focuses on several key dimensions of market behavior: price dynamics, risk, intraday price behavior, trading activity, and clustering information.

- **Price Dynamics:** The absolute return $|R_{i,t}|$ captures the magnitude of the most recent price movement, reflecting short-term market shocks.
- **Risk (Volatility):** Rolling standard deviations $RV_{i,t}^{(k)} = \mathrm{sd}(R_{i,t-k+1}, \ldots, R_{i,t})$ for $k \in \{5, 10, 21\}$ capture how volatile the stock has been over short and medium-term windows.
- **Intraday Price Behavior:** To account for within-day price variation, the high–low range is defined as $HL_{i,t} = \frac{H_{i,t} - L_{i,t}}{C_{i,t}}$, where $H_{i,t}$ and $L_{i,t}$. Rolling averages are then computed as $\overline{HL}_{i,t}^{(k)} = \frac{1}{k} \sum_{j=0}^{k-1} HL_{i,t-j}$ for $k \in \{5, 21\}$, smoothing out daily noise and capturing persistent intraday turbulence.
- **Trading Activity:** Relative volume $RelVol_{i,t}^{(k)} = \frac{V_{i,t}}{\overline{V}_{i,t}^{(k)}}$ measures how much trading activity deviates from the stock's recent norm.
- **Cluster Information:** A binary indicator flags whether a stock belongs to the higher-volatility cluster identified in the clustering stage.

LSTM Model Architecture and Training

The input to the model is a tensor of shape (21 timesteps, 9 features). The architecture consists of an LSTM layer with 32 units, a dense layer of 16 units with ReLU activation, and a single output node, producing the predicted 21-day forward volatility with 5,921 trainable parameters in total. The model is kept intentionally small to reduce overfitting risk.

Because the nine features operate on different scales, all features and the target are standardized using the training set mean and standard deviation only, preventing any information from the validation or test periods from leaking into the model.

The model is compiled using the default Adam optimizer and uses mean squared error as the loss function. The model is trained for up to 100 epochs with a batch size of 64.

Early stopping with a patience of 5 epochs monitors validation loss and restores the best weights.

```
3244/3244 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step
```

Naive Baseline

The benchmark forecast assumes volatility persists unchanged from its most recently observed level: $\hat{Y}_{i,t}^{\text{naive}} = RV_{i,t}^{(21)}$.

Evaluation Metrics

For forecasting, model performance is evaluated on the unseen test set using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), the coefficient of determination (R^2), and Pearson correlation between predicted and actual volatility. These metrics are computed for both the LSTM and the naive baseline, allowing direct comparison of predictive accuracy.

Results

The results are presented in two parts: clustering structure and forecasting performance.

Clustering Results

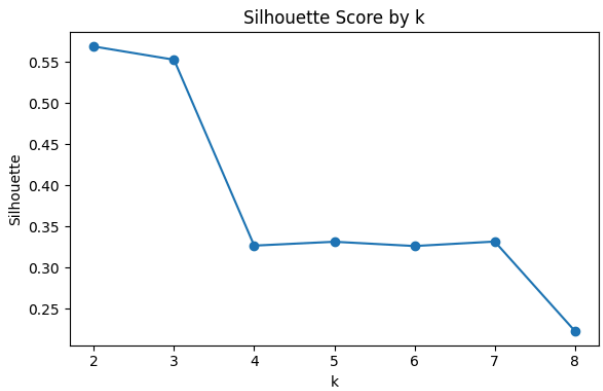
Selecting the Number of Clusters

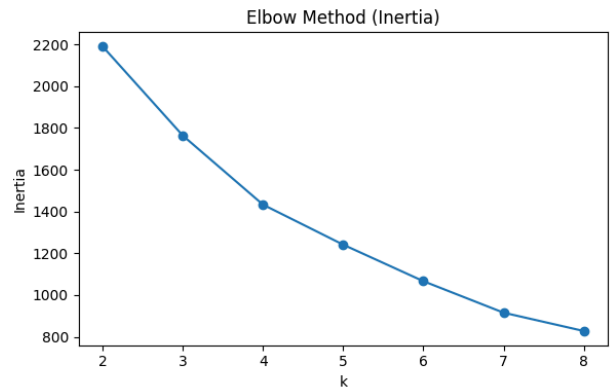
Three feature configurations were tested across 1-day, 5-day, and 21-day horizons. The 21-day configuration provided the most stable and interpretable clusters, as shorter horizons yielded noisier and less separable groupings. All subsequent results are based on this configuration.

Both the silhouette score and the elbow method indicate that $k = 2$ is the optimal number of clusters. The silhouette score peaks at 0.57 on the training set, drops modestly to 0.53 on the validation set, and falls further to 0.37 on the test set. This decline is expected because the clustering model is fit on training data only and applied to later periods using fixed centroids. As stocks shift their behavioral profiles over time, they naturally fit their assigned clusters less cleanly out of sample.

Feature Case Summary for Selecting Number of Clusters

Case	Return Feature	Volatility Feature	Volume Feature	N Stocks	Aggregated Feature Count	Best k	Best Train Silhouette
case_3	logret_21	logvol_21	vol_chg_21	499	6	2	0.569
case_2	logret_5	logvol_5	vol_chg_5	499	6	2	0.556
case_1	logret_1	logvol_1	vol_chg_1	499	6	2	0.528





Out[45]:

	split	silhouette
0	train	0.569157
1	validation	0.525816
2	test	0.374775

Cluster Profiles

The clustering provides two unequal groups: Cluster 0 contains 448 stocks, and Cluster 1 contains 51 stocks. Despite this imbalance, the clusters exhibit clear differences in behavior.

- Cluster 0 (lower-volatility group) exhibits positive average returns, with $\mathrm{logret_21} \approx 0.007$, and relatively low volatility, with $\mathrm{logvol_21} \approx 0.017$.
- Cluster 1 (higher-volatility group) shows slightly negative average returns ($\mathrm{logret_21} \approx -0.001$) and significantly higher volatility levels ($\mathrm{logvol_21} \approx 0.032$).

In addition to higher average volatility, Cluster 1 also shows greater variability. The standard deviation of returns is approximately twice as large in Cluster 1 ($\mathrm{logret_21_std} \approx 0.154$) compared to Cluster 0 (≈ 0.078), indicating more unstable price dynamics. Similarly, volatility variability ($\mathrm{logvol_21_std}$) and volume fluctuation variability ($\mathrm{vol_chg_21_std}$) are substantially higher in Cluster 1.

These results confirm a meaningful separation between two risk profiles: Cluster 0 as the stable, lower-risk group with moderate returns and trading volume, and Cluster 1 as the higher-risk group characterized by negative returns, elevated volatility, and larger swings in trading activity. This aligns with standard financial intuition distinguishing defensive stocks from risky ones, and validates that the clustering approach captures economically meaningful structure in the data.

Cluster Profile Means (Training Set)

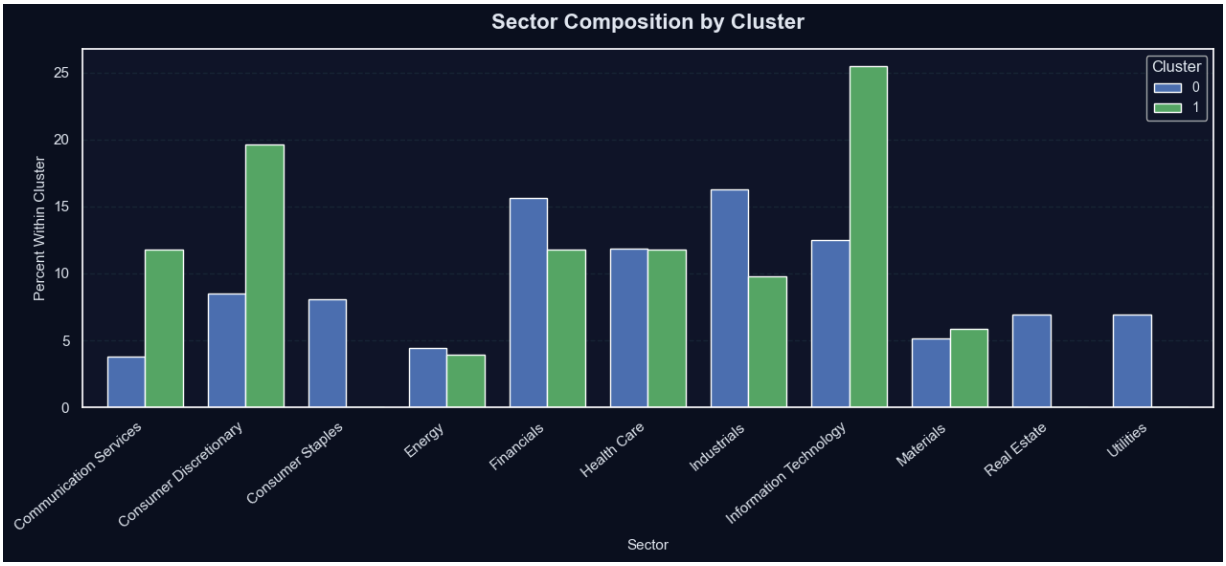
Metric	Cluster 0	Cluster 1
logret_21	0.006950	-0.001103
logvol_21	0.016940	0.031860
vol_chg_21	0.003339	0.006831
logret_21_std	0.077722	0.153690
logvol_21_std	0.005479	0.012880
vol_chg_21_std	0.530765	0.683821

Sector Composition

The sector breakdown reinforces the risk interpretation. Cluster 0 (lower-volatility group) is broadly diversified across sectors, with strong representation in Financials (70 stocks), Industrials (73 stocks), Health Care (53 stocks), and Information Technology (56 stocks). This suggests that more stable stocks are distributed across multiple sectors rather than concentrated in a single industry. In contrast, Cluster 1 (higher-volatility group) is smaller and more concentrated, with a relatively higher proportion of stocks in Information Technology (13) and Consumer Discretionary (10). The IT and Consumer Discretionary sectors are typically associated with higher growth potential but also greater uncertainty. Notably, sectors such as Consumer Staples, Real Estate, and Utilities have little to no representation in this cluster, consistent with their traditionally defensive, lower-risk nature.

Sector Counts by Cluster

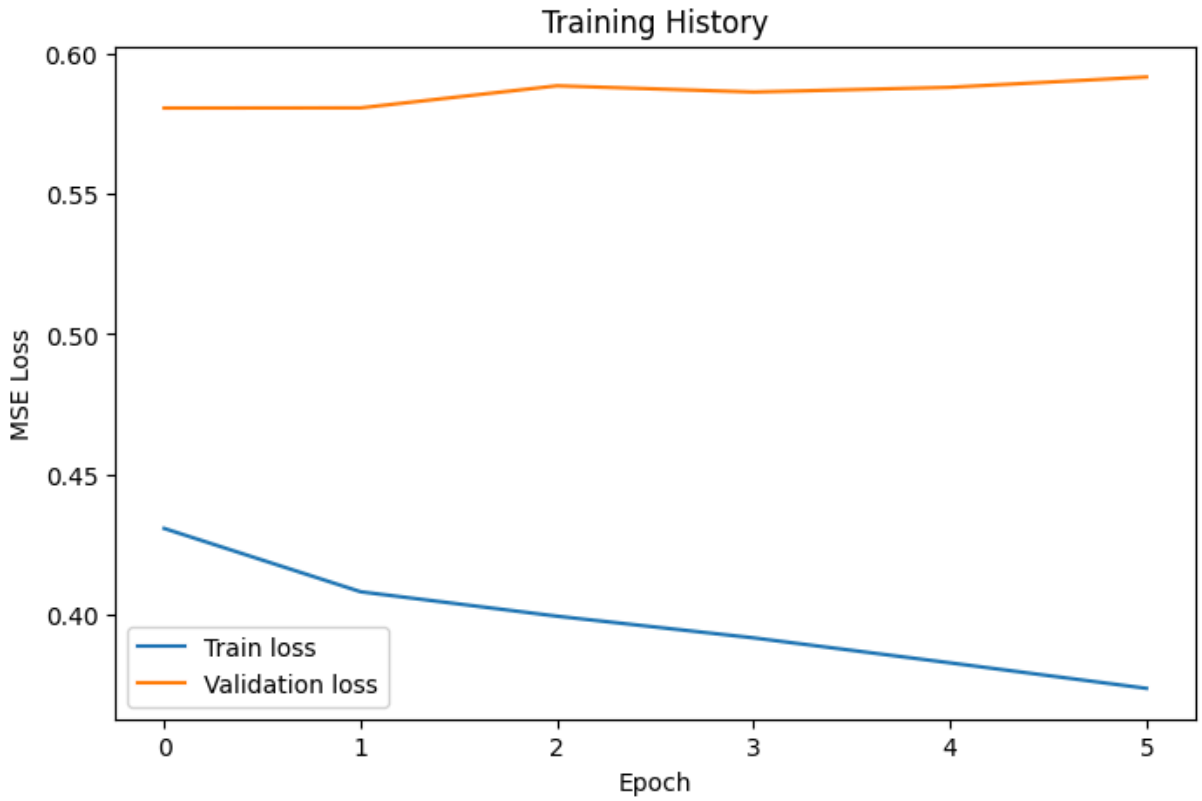
Sector	Cluster 0	Cluster 1
Communication Services	17	6
Consumer Discretionary	38	10
Consumer Staples	36	0
Energy	20	2
Financials	70	6
Health Care	53	6
Industrials	73	5
Information Technology	56	13
Materials	23	3
Real Estate	31	0
Utilities	31	0



LSTM Results

Training History

The Training History plot shows that training loss decreased steadily across epochs while validation loss plateaued early, with early stopping halting training at epoch 5. This points to overfitting, more specifically, the model learned the training data well but struggled to fully generalize. The relatively small architecture (5,921 parameters) was intentional to limit this, but the complexity of cross-sectional financial data across 499 stocks still presented a generalization challenge.



Forecasting Performance

The LSTM model is evaluated against the naive baseline on the unseen test set (March 2025 to March 2026). The Unseen Comparison (LSTM vs Baseline) table summarizes the results.

The LSTM outperforms the baseline across all four metrics. The 28% reduction in RMSE and 26% reduction in MAE indicate meaningfully improved prediction accuracy. More importantly, the baseline produces a negative R^2 of -0.2746, meaning it performs worse than simply predicting the mean volatility for every stock. The LSTM's positive R^2 of 0.338 shows it explains a real portion of the variation in future volatility, though substantial uncertainty remains.

```
3260/3260 ━━━━━━━━━━━━━━━━━━━━ 5s 1ms/step
```

Unseen Comparison (LSTM vs Baseline)

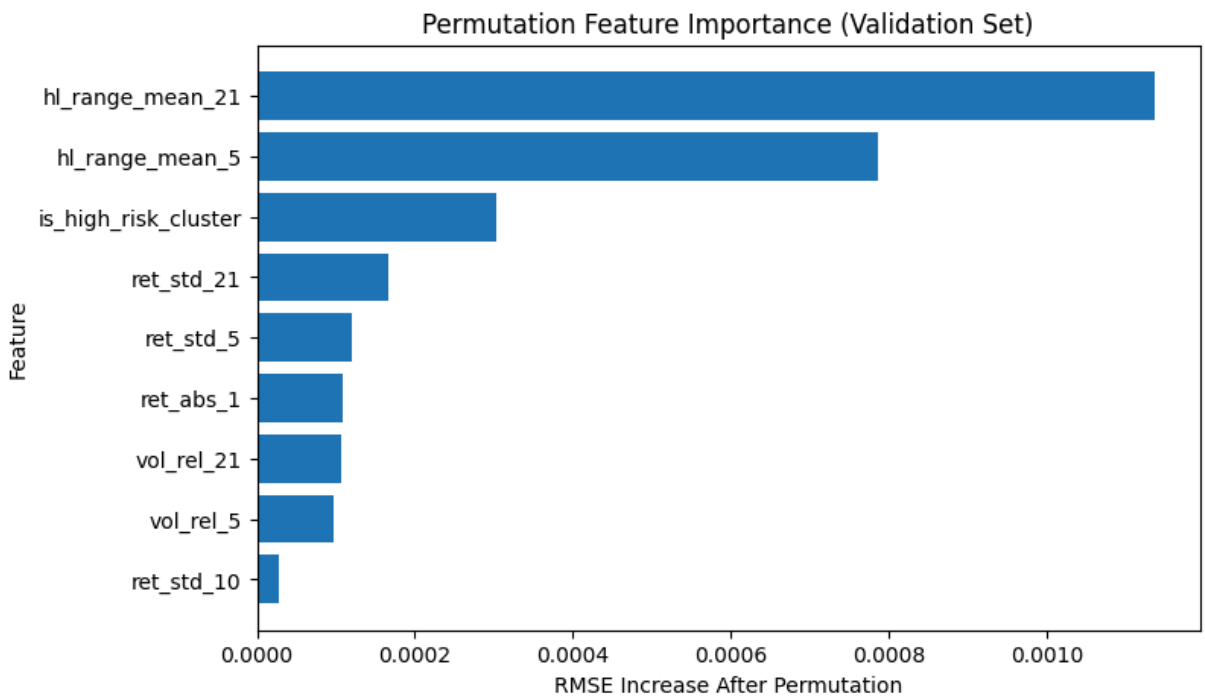
Model	RMSE	MAE	R2	Correlation
LSTM	0.0075	0.0051	0.3380	0.5842
Baseline	0.0104	0.0069	-0.2746	0.4195

Feature Importance

Feature importance is assessed using permutation importance, in which each feature is shuffled one at a time across the validation set and the resulting RMSE increase is recorded. The larger the RMSE increase, the more that feature contributed to predictions.

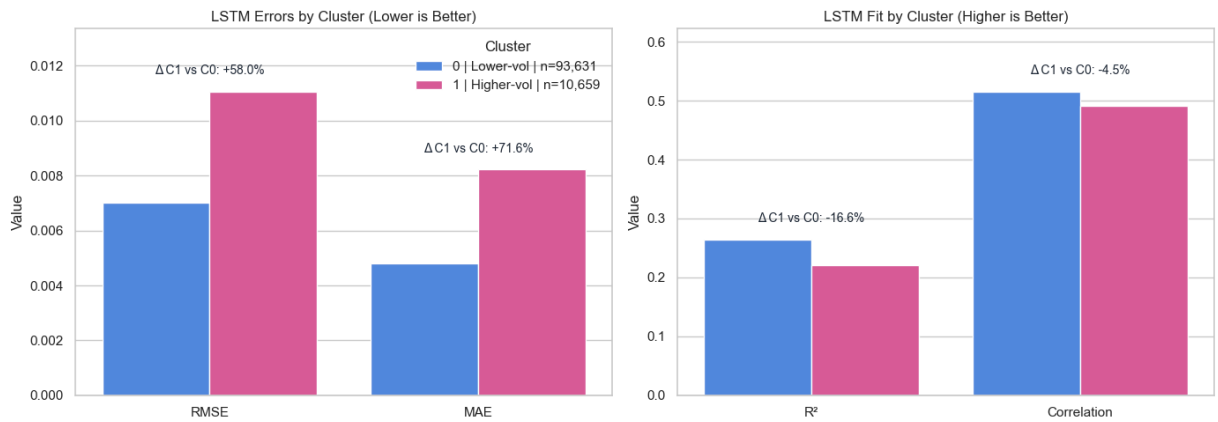
The Permutation Feature Importance (Validation Set) plot ranks the features by importance. The intra-day high–low range features (especially over the 21-day horizon) are the strongest predictors. This makes intuitive sense because how much a stock moves within a day reflects underlying uncertainty that closing prices alone do not capture. The cluster membership feature ranks third, ahead of all return-based and volume-based features, confirming that knowing a stock's risk profile adds real predictive value beyond market dynamics alone. Rolling volatility and relative volume features contribute modest but consistent signal.

```
Base metrics: {'rmse': np.float64(0.0072104052618797), 'mae': 0.004665113054
215908, 'corr': np.float64(0.6050221758770165)}
```



Performance Across Clusters

The model performed differently across the two clusters as shown in the comparison plot below. On the lower-volatility cluster, RMSE was 0.0070 and R^2 was 0.265. On the higher-volatility cluster, RMSE jumped to 0.0111 and R^2 fell to 0.221. These results indicate that the model produced larger prediction errors for the higher-volatility cluster, whereas it explained a greater proportion of variance in the lower-volatility cluster.

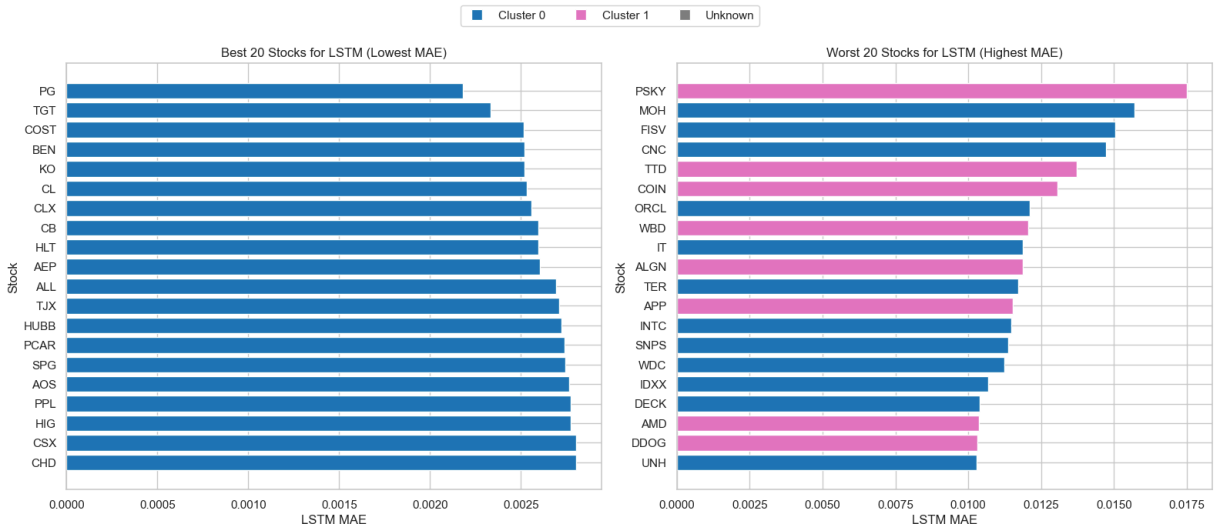


Stock-Level Performance

R^2 measures how much of the variance in actual volatility the model explains relative to simply predicting the mean. A positive R^2 indicates that the model is useful for that stock, as it predicts better than a flat average. As shown in the summary table below, only 48 out of 448 stocks in Cluster 0 achieved positive R^2, meaning the model was genuinely useful for only 10.7% of lower-volatility stocks. In Cluster 1, 13 out of 51 stocks achieved positive R^2, a rate of 25.5%. The higher percentage in Cluster 1 should not be over-interpreted because Cluster 1 has much smaller sample size. These results suggest that the model is only effective for a minority of stocks and does not generalize well across all 499 stocks.

	cluster	Cluster_name	n_stocks	n_r2_positive	pct_r2_positive
0	0	Lower-vol	448	48	10.714286
1	1	Higher-vol	51	13	25.490196

The below plot ranks MAE by stock and colors by cluster membership. Stocks with the highest predictive accuracy are concentrated in Cluster 0, while the lowest-accuracy stocks are distributed across both Cluster 0 and Cluster 1. This suggests that model performance is not uniformly driven by cluster membership; instead, Cluster 0 contains more stocks that are easier for the model to predict, but low performance cases exist in both clusters, indicating that other stock-specific factors beyond clustering likely play a significant role in prediction error.



Conclusions

Overall, K-means clustering successfully separated 499 S&P 500 stocks into two meaningful groups reflecting their risk profiles. The addition of cluster membership increased predictive value and improved volatility forecasting accuracy, directly addressing both research questions.

However, the model explains only 33.8% of the variance in future volatility on the test set. Most volatility remains unexplained, highlighting the inherent difficulty of forecasting financial markets. The high-low range being the strongest predictor raises a feature engineering question. Since high-low related features are used in their raw forms without log transformation or standardization, other features may have had their variability compressed during preprocessing, potentially limiting their contribution. Alternative feature representations and further tuning could be explored to address this.

The model also showed early signs of overfitting, with training halting at epoch 5 and a persistent gap between training and validation loss. The model learned patterns specific to 2021-2024 and may be less adaptable to structural shifts in later periods. Periodic retraining or a rolling window approach could improve adaptability to changing market conditions.

From a practical standpoint, the ability to distinguish between stable and high-risk stocks has real implications for risk modeling and portfolio management. Future research could explore richer feature sets incorporating macroeconomic indicators or earnings data, and more granular clustering with more than two groups to capture finer distinctions in stock behavior.

References

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

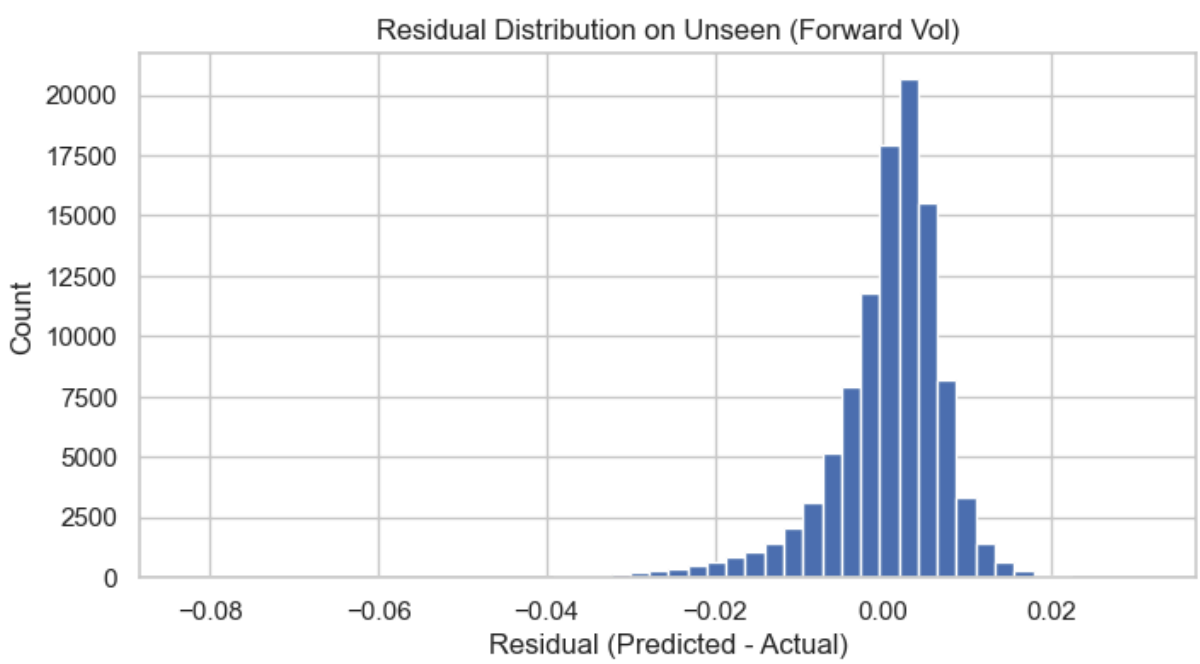
Scikit-learn Developers. (2024). *KMeans clustering*. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

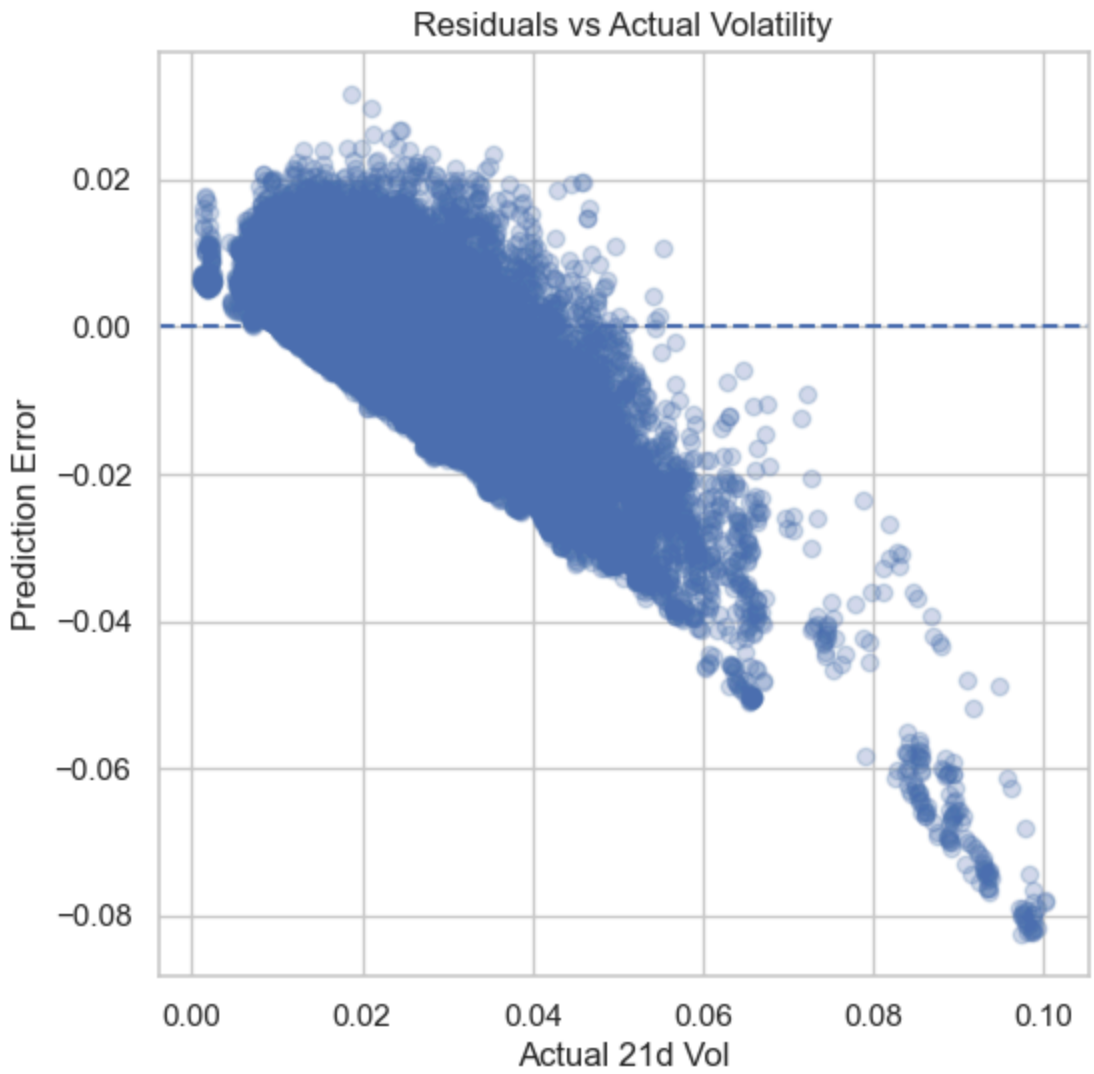
Scikit-learn Developers. (2024). *Permutation feature importance*. https://scikit-learn.org/stable/modules/permutation_importance.html

SR. (2024, September 13). Python for machine learning-powered volatility forecasting. Medium. https://medium.com/@deepml1818/volatility-forecasting-is-crucial-in-quantitative-finance-as-it-directly-affects-risk-management-2a9fd6387c2b

TensorFlow Developers. (2024). *Long short-term memory (LSTM) layer*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM

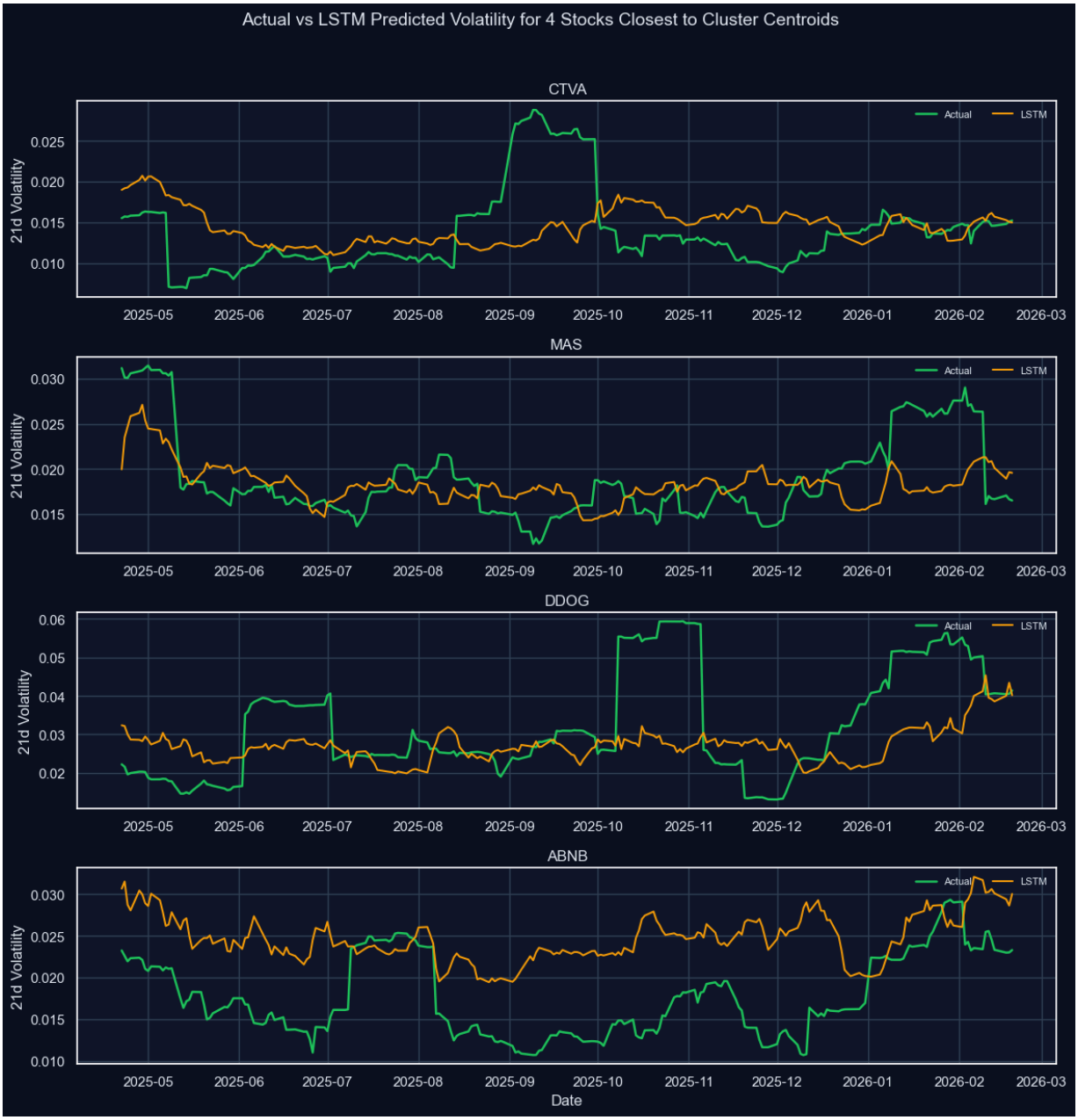
Appendix I





Closest Stocks to Cluster Centroids (Training Set)

Symbol	logret_21	logvol_21	vol_chg_21	logret_21_std	logvol_21_std	vol_chg_21_st
CTVA	0.005237	0.016873	0.005821	0.072331	0.005710	0.52825
MAS	0.007828	0.017548	0.007436	0.079766	0.005340	0.52236
DDOG	0.010838	0.036327	0.000362	0.151356	0.013394	0.62766
ABNB	-0.003440	0.029775	0.010216	0.130475	0.009681	0.67754


Actual vs LSTM Predicted Volatility for 4 Stocks Closest to Cluster Centroids
CTVA
MAS
DDOG
ABNB
Actual
LSTM
21d Volatility
Date
2025-05
2025-06
2025-07
2025-08
2025-09
2025-10
2025-11
2025-12
2026-01
2026-02
2026-03

Appendix 2

```
===== CODE APPENDIX =====

import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
e
from tensorflow import keras
from tensorflow.keras import layers

import matplotlib.pyplot as plt

def project_data_path(filename):
    for base in [Path.cwd(), *Path.cwd().parents]:
        candidate = base / 'data' / filename
        if candidate.exists():
            return candidate
    raise FileNotFoundError(f"Could not find data/{filename} from {Path.cwd
()}")

--------------------------------------------------

## CLUSTERING CODE ##

csv_path = project_data_path("sp500_5yr_with_sectors_weights.csv")

df = pd.read_csv(csv_path)
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values(['Symbol', 'Date']).copy()

def create_features(df):
    # Sort first so lag/rolling operations follow true chronological order p
er symbol
    df = df.sort_values(['Symbol', 'Date']).copy()
    df['log_close'] = np.log(df['Close'])

    # Returns
    df['logret_1'] = df.groupby('Symbol')['log_close'].diff(1)
    df['logret_5'] = df.groupby('Symbol')['log_close'].diff(5)
    df['logret_21'] = df.groupby('Symbol')['log_close'].diff(21)

    # Matched volatility windows
    df['logvol_1'] = df.groupby('Symbol')['logret_1'].rolling(2).std().reset
_index(level=0, drop=True)
    df['logvol_5'] = df.groupby('Symbol')['logret_1'].rolling(5).std().reset
_index(level=0, drop=True)
    df['logvol_21'] = df.groupby('Symbol')['logret_1'].rolling(21).std().res
et_index(level=0, drop=True)

    # Volume features
```

```
    df['log_volume'] = np.log(df['Volume'].clip(lower=1))

    # Keep original single-horizon volume change for appendix comparison
    df['vol_chg'] = df.groupby('Symbol')['log_volume'].diff(1)

    # Main pipeline uses horizon-matched volume-change features
    df['vol_chg_1'] = df.groupby('Symbol')['log_volume'].diff(1)
    df['vol_chg_5'] = df.groupby('Symbol')['log_volume'].diff(5)
    df['vol_chg_21'] = df.groupby('Symbol')['log_volume'].diff(21)

    return df

# split FIRST using your exact dates
train = df[(df['Date'] >= '2021-03-22') & (df['Date'] <= '2024-03-21')].copy
()
val   = df[(df['Date'] >= '2024-03-22') & (df['Date'] <= '2025-03-21')].copy
()
test  = df[(df['Date'] >= '2025-03-22') & (df['Date'] <= '2026-03-21')].copy
()

# THEN feature engineering
train = create_features(train)
val   = create_features(val)
test  = create_features(test)

--------------------------------------------------

common_symbols = (
    set(train['Symbol'].unique())
    & set(val['Symbol'].unique())
    & set(test['Symbol'].unique())
)

train = train[train['Symbol'].isin(common_symbols)].copy()
val   = val[val['Symbol'].isin(common_symbols)].copy()
test  = test[test['Symbol'].isin(common_symbols)].copy()

FEATURE_CASES = {
    "case_1": ['logret_1', 'logvol_1', 'vol_chg_1'],
    "case_2": ['logret_5', 'logvol_5', 'vol_chg_5'],
    "case_3": ['logret_21', 'logvol_21', 'vol_chg_21']
}

def aggregate_features(df, feature_cols):
    # Mean of all selected features
    agg_mean = df.groupby('Symbol')[feature_cols].mean()
    # Standard deviation only for time-varying features
    cols_for_std = list(feature_cols)
    agg_std = df.groupby('Symbol')[cols_for_std].std()
    agg_std.columns = [f"{col}_std" for col in agg_std.columns]

    # Combine mean + std
    X = pd.concat([agg_mean, agg_std], axis=1)

    return X.dropna()
```

```
--------------------------------------------------

def fit_kmeans(X, k):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)

    return model, scaler, labels, score, X_scaled

def apply_kmeans(X, model, scaler):
    X_scaled = scaler.transform(X)
    labels = model.predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    return labels, score, X_scaled

def evaluate_k_range(X, k_range=range(2, 9)):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    rows = []
    for k in k_range:
        model = KMeans(n_clusters=k, random_state=42, n_init=20)
        labels = model.fit_predict(X_scaled)
        sil = silhouette_score(X_scaled, labels)
        inertia = model.inertia_

        rows.append({
            "k": k,
            "silhouette": sil,
            "inertia": inertia
        })

    return pd.DataFrame(rows), X_scaled

--------------------------------------------------

feature_case_results = []

for case_name, case_features in FEATURE_CASES.items():
    X_train_case = aggregate_features(train, case_features)
    k_results_case, _ = evaluate_k_range(X_train_case, k_range=range(2, 9))

    best_idx = k_results_case['silhouette'].idxmax()
    best_k_case = int(k_results_case.loc[best_idx, 'k'])
    best_sil_case = float(k_results_case.loc[best_idx, 'silhouette'])

    feature_case_results.append({
        "case": case_name,
        "features": ", ".join(case_features),
        "n_stocks": len(X_train_case),
        "aggregated_feature_count": X_train_case.shape[1],
        "best_k": best_k_case,
        "best_train_silhouette": best_sil_case
```

```
    })

feature_case_results_df = (
    pd.DataFrame(feature_case_results)
    .sort_values("best_train_silhouette", ascending=False)
    .reset_index(drop=True)
)

--------------------------------------------------

## FORECASTING CODE ##

# load the data
csv_path = project_data_path("sp500_5yr_with_sectors_weights.csv")

df = pd.read_csv(csv_path)

if "Weight" in df.columns:
    df = df.drop(columns=["Weight"])

df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values(["Symbol", "Date"]).reset_index(drop=True)

# df.head()

cluster_map = pd.read_csv(project_data_path("sp500_stock_cluster_map.csv"))
df = df.merge(cluster_map[["Symbol", "cluster", "Cluster_name"]], on="Symbo
l", how="left")

df["is_high_risk_cluster"] = (df["cluster"] == 1).astype(int)

# time periods for splitting

TRAIN_START = pd.Timestamp("2021-03-22")
TRAIN_END   = pd.Timestamp("2024-03-20")

VAL_START   = pd.Timestamp("2024-03-21")
VAL_END     = pd.Timestamp("2025-03-20")

UNSEEN_START = pd.Timestamp("2025-03-21")
UNSEEN_END   = pd.Timestamp("2026-03-20")

# keep only stocks that have data in all 3 periods to avoid unseen stock iss
ues

train_symbols = set(df.loc[(df["Date"] >= TRAIN_START) & (df["Date"] <= TRAI
N_END), "Symbol"].unique())
val_symbols = set(df.loc[(df["Date"] >= VAL_START) & (df["Date"] <= VAL_EN
D), "Symbol"].unique())
unseen_symbols = set(df.loc[(df["Date"] >= UNSEEN_START) & (df["Date"] <= UN
SEEN_END), "Symbol"].unique())

common_symbols = sorted(train_symbols & val_symbols & unseen_symbols)

df = df[df["Symbol"].isin(common_symbols)].copy()
df = df.sort_values(["Symbol", "Date"]).reset_index(drop=True)
```

```
# print("Common stocks:", df["Symbol"].nunique())

# split into train/val/unseen

train_df = df[(df["Date"] >= TRAIN_START) & (df["Date"] <= TRAIN_END)].copy
()
val_df = df[(df["Date"] >= VAL_START) & (df["Date"] <= VAL_END)].copy()
unseen_df = df[(df["Date"] >= UNSEEN_START) & (df["Date"] <= UNSEEN_END)].co
py()

# export and remove unseen
unseen_df.to_csv(project_data_path("unseen_holdout_raw.csv"), index=False)

# combine train + val for modeling (will split later)
df = pd.concat([train_df, val_df], axis=0).sort_values(["Symbol", "Date"]).r
eset_index(drop=True)

# print("Model dataframe shape:", df.shape)
# print("Train + Val Date range:", df["Date"].min(), "to", df["Date"].max())

df_raw_dev = df.copy()

--------------------------------------------------

# feature engineering function

import numpy as np

def engineer_features(panel_df):
    panel_df = panel_df.copy()
    panel_df = panel_df.sort_values(["Symbol", "Date"])

    g = panel_df.groupby("Symbol")

    # simple 1 day return
    panel_df["ret_1"] = g["Close"].pct_change()

    # target: std of stock's daily returns from t+1 to t+21
    panel_df["target_vol_21d"] = (
        g["ret_1"].transform(lambda s: s.shift(-1).rolling(21).std().shift(-
20))
    )

    # today's absolute return
    panel_df["ret_abs_1"] = panel_df["ret_1"].abs()

    # std of returns over past 5, 10, 21 days
    for w in [5, 10, 21]:
        panel_df[f"ret_std_{w}"] = g["ret_1"].transform(lambda s: s.rolling
(w).std())

    # daily intraday range
    panel_df["hl_range"] = (panel_df["High"] - panel_df["Low"]) / panel_df
["Close"]
```

```
    for w in [5, 21]:
        panel_df[f"hl_range_mean_{w}"] = g["hl_range"].transform(lambda s:
s.rolling(w).mean())

    # relative volume
    for w in [5, 21]:
        panel_df[f"vol_mean_{w}"] = g["Volume"].transform(lambda s: s.rollin
g(w).mean())
        panel_df[f"vol_rel_{w}"] = panel_df["Volume"] / panel_df[f"vol_mean_
{w}"]

    return panel_df

# apply feature engineering to dev set
df = engineer_features(df)

feature_cols = [
    "ret_abs_1",
    "ret_std_5",
    "ret_std_10",
    "ret_std_21",
    "hl_range_mean_5",
    "hl_range_mean_21",
    "vol_rel_5",
    "vol_rel_21",
    "is_high_risk_cluster"
]

target_col = "target_vol_21d"

--------------------------------------------------

# sequence dataset construction

# Build modeling rows where both sequence history and forward target window
are valid.

LOOKBACK = 21
HORIZON = 21

# Keep only finite rows once before split filtering.
df_feat = df.dropna(subset=feature_cols + [target_col]).copy()
finite_mask = np.isfinite(df_feat[feature_cols + [target_col]]).all(axis=1)
df_feat = df_feat.loc[finite_mask].copy()

# Endpoint date t must be in split, and target window (t+1 ... t+21) must st
ay in same split.
df_feat = df_feat.sort_values(["Symbol", "Date"]).copy()
g = df_feat.groupby("Symbol")

# Per-symbol forward availability guard.
df_feat["date_plus_horizon"] = g["Date"].shift(-HORIZON)

train_feat = df_feat[
    (df_feat["Date"] >= TRAIN_START) &
    (df_feat["Date"] <= TRAIN_END) &
```

```
    (df_feat["date_plus_horizon"].notna()) &
    (df_feat["date_plus_horizon"] <= TRAIN_END)
].copy()

val_feat = df_feat[
    (df_feat["Date"] >= VAL_START) &
    (df_feat["Date"] <= VAL_END) &
    (df_feat["date_plus_horizon"].notna()) &
    (df_feat["date_plus_horizon"] <= VAL_END)
].copy()

# print("train_feat:", train_feat.shape)
# print("val_feat:", val_feat.shape)

# sequence construction function
def make_sequences(panel_df, feature_cols, target_col, lookback=21):
    X_list = []
    y_list = []
    meta_list = []

    panel_df = panel_df.sort_values(["Symbol", "Date"]).copy()

    for symbol, g in panel_df.groupby("Symbol"):
        g = g.sort_values("Date").reset_index(drop=True)

        X_values = g[feature_cols].to_numpy(dtype=np.float32)
        y_values = g[target_col].to_numpy(dtype=np.float32)
        dates = g["Date"].to_numpy()

        for t in range(lookback - 1, len(g)):
            X_window = X_values[t - lookback + 1 : t + 1]
            y_target = y_values[t]
            end_date = dates[t]

            if not np.isfinite(X_window).all():
                continue
            if not np.isfinite(y_target):
                continue

            X_list.append(X_window)
            y_list.append(y_target)
            meta_list.append({
                "Symbol": symbol,
                "Date": end_date
            })

    X = np.array(X_list, dtype=np.float32)
    y = np.array(y_list, dtype=np.float32)
    meta = pd.DataFrame(meta_list)

    return X, y, meta

--------------------------------------------------

# train sequences
```

```
# Train sequences
X_train, y_train, meta_train = make_sequences(
    train_feat,
    feature_cols=feature_cols,
    target_col=target_col,
    lookback=LOOKBACK
)

meta_train["Date"] = pd.to_datetime(meta_train["Date"])

# print("Train:", X_train.shape, y_train.shape)

# Validation needs trailing train context so earliest validation windows are
complete.
train_context = (
    train_feat
    .sort_values(["Symbol", "Date"])
    .groupby("Symbol", group_keys=False)
    .tail(LOOKBACK - 1)
    .copy()
)

val_with_context = (
    pd.concat([train_context, val_feat], axis=0)
    .sort_values(["Symbol", "Date"])
    .drop_duplicates(subset=["Symbol", "Date"], keep="last")
    .reset_index(drop=True)
)

X_val_all, y_val_all, meta_val_all = make_sequences(
    val_with_context,
    feature_cols=feature_cols,
    target_col=target_col,
    lookback=LOOKBACK
)

meta_val_all["Date"] = pd.to_datetime(meta_val_all["Date"])

val_keep = (
    (meta_val_all["Date"] >= VAL_START) &
    (meta_val_all["Date"] <= VAL_END)
)

X_val = X_val_all[val_keep.values]
y_val = y_val_all[val_keep.values]
meta_val = meta_val_all.loc[val_keep].reset_index(drop=True)

# print("Val:", X_val.shape, y_val.shape)

--------------------------------------------------

# scale features

# scale features and target using train only

n_features = X_train.shape[2]
```

```
train_2d = X_train.reshape(-1, n_features)
feature_means = train_2d.mean(axis=0)
feature_stds = train_2d.std(axis=0) + 1e-8

X_train_scaled = (X_train - feature_means) / feature_stds
X_val_scaled = (X_val - feature_means) / feature_stds

y_mean = y_train.mean()
y_std = y_train.std() + 1e-8

y_train_scaled = (y_train - y_mean) / y_std
y_val_scaled = (y_val - y_mean) / y_std

--------------------------------------------------

# LSTM training & validation

# build model

keras.utils.set_random_seed(42)

model = keras.Sequential([
    layers.Input(shape=(LOOKBACK, len(feature_cols))),
    layers.LSTM(32),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# model.summary()

# train

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    )
]

history = model.fit(
    X_train_scaled,
    y_train_scaled,
    validation_data=(X_val_scaled, y_val_scaled),
    epochs=100,
    batch_size=64,
    callbacks=callbacks,
    verbose=0
)
```

```
val_pred_scaled = model.predict(X_val_scaled).reshape(-1)
val_pred = val_pred_scaled * y_std + y_mean

val_results = meta_val.copy()
val_results["y_true"] = y_val
val_results["y_pred"] = val_pred

# val_results.head()

--------------------------------------------------

feature_case_results_df

--------------------------------------------------

best_case = feature_case_results_df.loc[0, 'case']
best_features = FEATURE_CASES[best_case]

kmeans_X_train = aggregate_features(train, best_features)
k_results, _ = evaluate_k_range(kmeans_X_train, k_range=range(2, 9))

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(k_results['k'], k_results['silhouette'], marker='o')
plt.title("Silhouette Score by k")
plt.xlabel("k")
plt.ylabel("Silhouette")

plt.subplot(1, 2, 2)
plt.plot(k_results['k'], k_results['inertia'], marker='o')
plt.title("Elbow Method (Inertia)")
plt.xlabel("k")
plt.ylabel("Inertia")

plt.tight_layout()
plt.show()

--------------------------------------------------

best_k = int(k_results.loc[k_results['silhouette'].idxmax(), 'k'])
kmeans_model, kmeans_scaler, train_labels, train_score, kmeans_X_train_scale
d = fit_kmeans(kmeans_X_train, k=best_k)

train_clusters = pd.DataFrame({
    "Symbol": kmeans_X_train.index,
    "Cluster_train": train_labels
})

kmeans_X_val = aggregate_features(val, best_features)

common_train_val = kmeans_X_train.index.intersection(kmeans_X_val.index)
kmeans_X_val = kmeans_X_val.loc[common_train_val]

val_labels, val_score, kmeans_X_val_scaled = apply_kmeans(kmeans_X_val, kmea
```

```
ns_model, kmeans_scaler)

val_clusters = pd.DataFrame({
    "Symbol": kmeans_X_val.index,
    "Cluster_val": val_labels
})

kmeans_X_test = aggregate_features(test, best_features)

common_train_test = kmeans_X_train.index.intersection(kmeans_X_test.index)
kmeans_X_test = kmeans_X_test.loc[common_train_test]

test_labels, test_score, kmeans_X_test_scaled = apply_kmeans(kmeans_X_test,
kmeans_model, kmeans_scaler)

test_clusters = pd.DataFrame({
    "Symbol": kmeans_X_test.index,
    "Cluster_test": test_labels
})

--------------------------------------------------

stability_summary = pd.DataFrame({
    "split": ["train", "validation", "test"],
    "silhouette": [train_score, val_score, test_score]
})

stability_summary

--------------------------------------------------

train_profile = X_train.copy()
train_profile["Cluster"] = train_labels

cluster_profile = train_profile.groupby("Cluster").mean()
cluster_profile

--------------------------------------------------

cluster_profile = train_profile.groupby("Cluster").mean()
cluster_profile

--------------------------------------------------

train_profile = X_train.copy()
train_profile["Cluster"] = train_labels
cluster_profile = train_profile.groupby("Cluster").mean()
cluster_profile

--------------------------------------------------

train_profile = kmeans_X_train.copy()
train_profile["Cluster"] = train_labels

cluster_profile = train_profile.groupby("Cluster").mean()
cluster_profile
```

```
--------------------------------------------------

train_profile = kmeans_X_train.copy()
train_profile["Cluster"] = train_labels

cluster_profile = train_profile.groupby("Cluster").mean()
cluster_profile

--------------------------------------------------

from scipy.spatial.distance import cdist

# distance to centroids (using scaled data)
dist_matrix = cdist(kmeans_X_train_scaled, kmeans_model.cluster_centers_)

# distance to assigned centroid
assigned_dist = [
    dist_matrix[i, label] for i, label in enumerate(train_labels)
]

# attach to dataframe
X_train_with_labels = kmeans_X_train.copy()
X_train_with_labels['cluster'] = train_labels
X_train_with_labels['distance_to_centroid'] = assigned_dist

# get 2 closest stocks per cluster
closest_stocks = (
    X_train_with_labels
    .sort_values(['cluster', 'distance_to_centroid'])
    .groupby('cluster')
    .head(2)
 )

# closest_stocks

--------------------------------------------------

import matplotlib.pyplot as plt

# Sector concentration test: are clusters economically interpretable?
if "Sector" in train.columns:
    sector_col = "Sector"
elif "GICS Sector" in train.columns:
    sector_col = "GICS Sector"
else:
    sector_col = None

if sector_col is None:
    print("No sector column found (expected 'Sector' or 'GICS Sector').")
else:
    symbol_sector = (
        train[["Symbol", sector_col]]
        .dropna()
        .drop_duplicates(subset=["Symbol"])
        .rename(columns={sector_col: "Sector"})
```

```
    )

    # stock-level cluster assignments
    train_clusters = X_train_with_labels[["cluster"]].reset_index()

    cluster_sector = (
        train_clusters
        .merge(symbol_sector, on="Symbol", how="left")
        .dropna(subset=["Sector"])
    )

    sector_counts = pd.crosstab(cluster_sector["cluster"], cluster_sector["S
ector"])
    sector_pct = sector_counts.div(sector_counts.sum(axis=1), axis=0) * 100

    print("Sector counts by cluster")
    display(sector_counts)

    #print("Sector percentages within each cluster")
    #display(sector_pct.round(2))

    # Default Matplotlib style (no custom dark theme)
    cluster_colors = ["#4C72B0", "#55A868", "#C44E52", "#8172B2"]

    fig, ax = plt.subplots(figsize=(13, 6))

    sector_pct.T.plot(
        kind="bar",
        ax=ax,
        color=cluster_colors[:len(sector_pct.index)],
        width=0.75
    )

    ax.set_title(
        "Sector Composition by Cluster",
        fontsize=16,
        fontweight="bold",
        pad=15
    )
    ax.set_ylabel("Percent Within Cluster", fontsize=11)
    ax.set_xlabel("Sector", fontsize=11)

    ax.tick_params(axis="x", rotation=40)
    ax.grid(axis="y", linestyle="--", alpha=0.35)
    ax.grid(axis="x", visible=False)

    ax.legend(title="Cluster", fontsize=10)

    plt.xticks(ha="right")
    plt.tight_layout()
    plt.show()

--------------------------------------------------

# Validation metrics for the LSTM.
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
e
import numpy as np

rmse = np.sqrt(mean_squared_error(val_results["y_true"], val_results["y_pre
d"]))
mae = mean_absolute_error(val_results["y_true"], val_results["y_pred"])
r2 = r2_score(val_results["y_true"], val_results["y_pred"])
corr = val_results["y_true"].corr(val_results["y_pred"])

# print(f"Validation RMSE: {rmse:.4f}")
# print(f"Validation MAE: {mae:.4f}")
# print(f"Validation R2: {r2:.4f}")
# print(f"Validation correlation: {corr:.4f}")  # directional tracking

# plot training curve

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Train loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.title("Training History")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()

--------------------------------------------------

# test on unseen holdout

# Build unseen panel with trailing dev context, then engineer features on th
e combined raw panel.

unseen_df = pd.read_csv("/Users/mintie/Desktop/Multivariate_Project-1/data/u
nseen_holdout_raw.csv")

unseen_df["Date"] = pd.to_datetime(unseen_df["Date"])
unseen_df = unseen_df.sort_values(["Symbol", "Date"]).reset_index(drop=True)

# print(unseen_df["Date"].min(), unseen_df["Date"].max())
# print(unseen_df.shape)
# print("Stocks:", unseen_df["Symbol"].nunique())

dev_context_raw = (
    df_raw_dev
    .sort_values(["Symbol", "Date"])
    .groupby("Symbol", group_keys=False)
    .tail(LOOKBACK - 1)
    .copy()
)

unseen_raw_with_context = (
    pd.concat([dev_context_raw, unseen_df], axis=0)
    .sort_values(["Symbol", "Date"])
    .drop_duplicates(subset=["Symbol", "Date"], keep="last")
    .reset_index(drop=True)
```

```
)

unseen_feat_all = engineer_features(unseen_raw_with_context)

unseen_feat = unseen_feat_all[
    (unseen_feat_all["Date"] >= UNSEEN_START) &
    (unseen_feat_all["Date"] <= UNSEEN_END)
].copy()

unseen_feat = unseen_feat.dropna(subset=feature_cols + [target_col]).copy()
finite_mask = np.isfinite(unseen_feat[feature_cols + [target_col]]).all(axis
=1)
unseen_feat = unseen_feat.loc[finite_mask].copy()

# print("unseen_feat:", unseen_feat.shape)

--------------------------------------------------

# unseen sequences from engineered panel that already includes raw dev conte
xt
X_unseen_all, y_unseen_all, meta_unseen_all = make_sequences(
    unseen_feat_all,
    feature_cols=feature_cols,
    target_col=target_col,
    lookback=LOOKBACK
)

meta_unseen_all["Date"] = pd.to_datetime(meta_unseen_all["Date"])

unseen_keep = (
    (meta_unseen_all["Date"] >= UNSEEN_START) &
    (meta_unseen_all["Date"] <= UNSEEN_END)
)

X_unseen = X_unseen_all[unseen_keep.values]
y_unseen = y_unseen_all[unseen_keep.values]
meta_unseen = meta_unseen_all.loc[unseen_keep].reset_index(drop=True)

# print("Unseen:", X_unseen.shape, y_unseen.shape)

--------------------------------------------------

# Unseen evaluation: LSTM vs Baseline (past 21d volatility)

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
e
import numpy as np
import pandas as pd
from IPython.display import display

# Scale unseen features using train stats only
X_unseen_scaled = (X_unseen - feature_means) / feature_stds

# Predict with LSTM and invert target scaling
unseen_pred_scaled = model.predict(X_unseen_scaled).reshape(-1)
unseen_pred = unseen_pred_scaled * y_std + y_mean
```

```
# Build unseen results table (model side)
unseen_results = meta_unseen.copy()
unseen_results["y_true"] = y_unseen
unseen_results["y_pred"] = unseen_pred

# Add baseline prediction (naive = ret_std_21)
baseline_map_unseen = (
    unseen_feat[["Symbol", "Date", "ret_std_21"]]
    .drop_duplicates(["Symbol", "Date"])
    .copy()
 )

unseen_eval = unseen_results.merge(
    baseline_map_unseen,
    on=["Symbol", "Date"],
    how="left"
)
unseen_eval["baseline_pred"] = unseen_eval["ret_std_21"]

# Evaluate on same rows for fair comparison
eval_df = unseen_eval.dropna(subset=["y_true", "y_pred", "baseline_pred"]).c
opy()

def metric_row(y_true, y_hat):
    return {
        "RMSE": np.sqrt(mean_squared_error(y_true, y_hat)),
        "MAE": mean_absolute_error(y_true, y_hat),
        "R2": r2_score(y_true, y_hat),
        "Correlation": pd.Series(y_true).corr(pd.Series(y_hat)),
    }

comparison = pd.DataFrame(
    {
        "LSTM": metric_row(eval_df["y_true"], eval_df["y_pred"]),
        "Baseline": metric_row(eval_df["y_true"], eval_df["baseline_pred"]),
    }
).T.round(4)

# Table output
comparison_table = comparison.copy()
comparison_table.index.name = "Model"
display(
    comparison_table.style
    .format(precision=4)
    .set_caption("Unseen Comparison (LSTM vs Baseline)")
    .set_table_styles([
        {"selector": "caption", "props": [("caption-side", "top"), ("font-we
ight", "bold")]},
    ])
 )

--------------------------------------------------

from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
```

```
import pandas as pd

def evaluate_model(model, X_scaled, y_true, y_mean, y_std):
    pred_scaled = model.predict(X_scaled, verbose=0).reshape(-1)
    pred = pred_scaled * y_std + y_mean

    rmse = np.sqrt(mean_squared_error(y_true, pred))
    mae = mean_absolute_error(y_true, pred)
    corr = pd.Series(y_true).corr(pd.Series(pred))

    return {"rmse": rmse, "mae": mae, "corr": corr}

# baseline validation performance
base_metrics = evaluate_model(model, X_val_scaled, y_val, y_mean, y_std)
print("Base metrics:", base_metrics)

# permutation importance
rng = np.random.default_rng(42)
importance_rows = []

for j, feature in enumerate(feature_cols):
    X_perm = X_val_scaled.copy()

    # shuffle this feature across samples, preserving the 21-step sequence w
ithin each sample
    perm_idx = rng.permutation(X_perm.shape[0])
    X_perm[:, :, j] = X_perm[perm_idx, :, j]

    perm_metrics = evaluate_model(model, X_perm, y_val, y_mean, y_std)

    importance_rows.append({
        "feature": feature,
        "rmse_increase": perm_metrics["rmse"] - base_metrics["rmse"],
        "mae_increase": perm_metrics["mae"] - base_metrics["mae"],
        "corr_drop": base_metrics["corr"] - perm_metrics["corr"],
    })

importance_df = pd.DataFrame(importance_rows).sort_values(
    "rmse_increase", ascending=False
)

import matplotlib.pyplot as plt

plot_df = importance_df.sort_values("rmse_increase", ascending=True)

plt.figure(figsize=(8, 5))
plt.barh(plot_df["feature"], plot_df["rmse_increase"])
plt.xlabel("RMSE Increase After Permutation")
plt.ylabel("Feature")
plt.title("Permutation Feature Importance (Validation Set)")
plt.show()

--------------------------------------------------

import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
e

# Build by_cluster_perf if it is not already defined in the kernel.
if "by_cluster_perf" not in globals():
    work = unseen_results.copy()

    if "cluster" not in work.columns or "Cluster_name" not in work.columns:
        work = work.merge(
            cluster_map[["Symbol", "cluster", "Cluster_name"]].drop_duplicat
es(subset=["Symbol"]),
            on="Symbol",
            how="left"
        )

    rows = []
    for (cluster, name), g in work.groupby(["cluster", "Cluster_name"], drop
na=False):
        g_clean = g.dropna(subset=["y_true", "y_pred"]).copy()
        if len(g_clean) == 0:
            continue
        rows.append({
            "cluster": cluster,
            "Cluster_name": name,
            "rmse_lstm": np.sqrt(mean_squared_error(g_clean["y_true"], g_cle
an["y_pred"])),
            "mae_lstm": mean_absolute_error(g_clean["y_true"], g_clean["y_pr
ed"]),
            "r2_lstm": r2_score(g_clean["y_true"], g_clean["y_pred"]),
            "corr_lstm": g_clean["y_true"].corr(g_clean["y_pred"]),
            "n_obs": len(g_clean),
        })

    by_cluster_perf = pd.DataFrame(rows)

df = by_cluster_perf.copy()
df = df[df["cluster"].isin([0, 1])].sort_values("cluster").reset_index(drop=
True)

if df["cluster"].nunique() != 2:
    raise ValueError("Need exactly 2 clusters (0 and 1) to compute % chang
e.")

df["cluster_label"] = (
    df["cluster"].astype(str)
    + " | "
    + df["Cluster_name"].astype(str)
    + " | n="
    + df["n_obs"].map("{:,}".format)
)

sns.set_theme(style="whitegrid")

cluster_order = df["cluster_label"].tolist()  # cluster 0 first, cluster 1 s
```

```
econd
cluster_palette = {
    cluster_order[0]: "#3B82F6",  # Cluster 0
    cluster_order[1]: "#EC4899",  # Cluster 1
}

err_order = ["RMSE", "MAE"]
fit_order = ["R²", "Correlation"]

err_long = df.melt(
    id_vars=["cluster_label"],
    value_vars=["rmse_lstm", "mae_lstm"],
    var_name="metric",
    value_name="value",
)
err_long["metric"] = err_long["metric"].map({
    "rmse_lstm": "RMSE",
    "mae_lstm": "MAE",
})

fit_long = df.melt(
    id_vars=["cluster_label"],
    value_vars=["r2_lstm", "corr_lstm"],
    var_name="metric",
    value_name="value",
)
fit_long["metric"] = fit_long["metric"].map({
    "r2_lstm": "R²",
    "corr_lstm": "Correlation",
})

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

def barplot_no_err(data, metric_order, ax):
    try:
        sns.barplot(
            data=data,
            x="metric",
            y="value",
            hue="cluster_label",
            order=metric_order,
            hue_order=cluster_order,
            palette=cluster_palette,
            errorbar=None,   # seaborn >= 0.12
            ax=ax,
        )
    except TypeError:
        sns.barplot(
            data=data,
            x="metric",
            y="value",
            hue="cluster_label",
            order=metric_order,
            hue_order=cluster_order,
            palette=cluster_palette,
            ci=None,         # seaborn < 0.12
```

```
            ax=ax,
        )

def add_pct_change_labels(ax, data_long, metric_order):
    p = data_long.pivot(index="metric", columns="cluster_label", values="val
ue")
    c0, c1 = cluster_order[0], cluster_order[1]

    y0, y1 = ax.get_ylim()
    span = y1 - y0
    ax.set_ylim(y0, y1 + 0.15 * span)
    y0, y1 = ax.get_ylim()
    span = y1 - y0

    for i, m in enumerate(metric_order):
        v0 = p.loc[m, c0]
        v1 = p.loc[m, c1]
        pct = np.nan if v0 == 0 else (v1 - v0) / v0 * 100.0
        txt = "Δ C1 vs C0: n/a" if np.isnan(pct) else f"Δ C1 vs C0: {pct:+.1
f}%"

        y_text = max(v0, v1) + 0.04 * span
        ax.text(i, y_text, txt, ha="center", va="bottom", fontsize=10, color
="#1F2937")

barplot_no_err(err_long, err_order, axes[0])
axes[0].set_title("LSTM Errors by Cluster (Lower is Better)")
axes[0].set_xlabel("")
axes[0].set_ylabel("Value")
axes[0].grid(axis="x", visible=False)
add_pct_change_labels(axes[0], err_long, err_order)

barplot_no_err(fit_long, fit_order, axes[1])
axes[1].axhline(0, color="#94A3B8", lw=1)
axes[1].set_title("LSTM Fit by Cluster (Higher is Better)")
axes[1].set_xlabel("")
axes[1].set_ylabel("Value")
axes[1].grid(axis="x", visible=False)
add_pct_change_labels(axes[1], fit_long, fit_order)

if axes[1].legend_ is not None:
    axes[1].legend_.remove()
axes[0].legend(title="Cluster", frameon=False, loc="upper right")

plt.tight_layout()
plt.show()

--------------------------------------------------

from sklearn.metrics import r2_score
import pandas as pd

min_obs_per_stock = 3

# —— FIX: bring cluster columns into unseen_results if not already there ——
if "cluster" not in unseen_results.columns:
```

```
    unseen_results = unseen_results.merge(
        cluster_map[["Symbol", "cluster", "Cluster_name"]],
        on="Symbol",
        how="left"
    )

d = unseen_results.dropna(subset=["Symbol", "y_true", "y_pred"]).copy()

rows = []
for sym, g in d.groupby("Symbol"):
    if len(g) < min_obs_per_stock:
        continue
    if g["y_true"].nunique() < 2:
        continue
    rows.append({
        "Symbol": sym,
        "r2_stock": r2_score(g["y_true"], g["y_pred"]),
        "cluster": g["cluster"].iloc[0],
        "Cluster_name": g["Cluster_name"].iloc[0],
    })

stock_r2 = pd.DataFrame(rows)

cluster_summary = (
    stock_r2.assign(r2_positive=stock_r2["r2_stock"] > 0)
    .groupby(["cluster", "Cluster_name"], dropna=False)
    .agg(
        n_stocks=("Symbol", "nunique"),
        n_r2_positive=("r2_positive", "sum")
    )
    .reset_index()
)

cluster_summary["pct_r2_positive"] = (
    100 * cluster_summary["n_r2_positive"] / cluster_summary["n_stocks"]
)

display(cluster_summary)

--------------------------------------------------

# Best/Worst LSTM stocks with bar color = cluster
# Blue = cluster 0, Pink = cluster 1

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
e

sns.set_style("whitegrid")

top_n = 20
min_obs = 50

# Build stock_level on the fly if it is not in memory yet.
```

```
if "stock_level" not in globals():
    work = unseen_results.copy()

    if "cluster" not in work.columns or "Cluster_name" not in work.columns:
        work = work.merge(
            cluster_map[["Symbol", "cluster", "Cluster_name"]].drop_duplicat
es(subset=["Symbol"]),
            on="Symbol",
            how="left"
        )

    # Add baseline predictions if unavailable.
    if "baseline_pred" not in work.columns and "ret_std_21" in work.columns:
        work["baseline_pred"] = work["ret_std_21"]

    stock_rows = []
    for sym, g in work.groupby("Symbol"):
        g_clean = g.dropna(subset=["y_true", "y_pred"]).copy()
        if len(g_clean) == 0:
            continue

        row = {
            "Symbol": sym,
            "cluster": g_clean["cluster"].iloc[0] if "cluster" in g_clean.co
lumns else np.nan,
            "Cluster_name": g_clean["Cluster_name"].iloc[0] if "Cluster_nam
e" in g_clean.columns else np.nan,
            "mae_lstm": mean_absolute_error(g_clean["y_true"], g_clean["y_pr
ed"]),
            "rmse_lstm": np.sqrt(mean_squared_error(g_clean["y_true"], g_cle
an["y_pred"])),
            "r2_lstm": r2_score(g_clean["y_true"], g_clean["y_pred"]) if g_c
lean["y_true"].nunique() > 1 else np.nan,
            "n_obs": len(g_clean),
        }

        if "baseline_pred" in g_clean.columns:
            g_b = g_clean.dropna(subset=["baseline_pred"]).copy()
            if len(g_b) > 0:
                row["mae_baseline"] = mean_absolute_error(g_b["y_true"], g_b
["baseline_pred"])
                row["rmse_baseline"] = np.sqrt(mean_squared_error(g_b["y_tru
e"], g_b["baseline_pred"]))
                row["r2_baseline"] = r2_score(g_b["y_true"], g_b["baseline_p
red"]) if g_b["y_true"].nunique() > 1 else np.nan

        stock_rows.append(row)

    stock_level = pd.DataFrame(stock_rows)

# Use the per-stock table.
plot_df = stock_level.copy()
plot_df = plot_df[plot_df["n_obs"] >= min_obs].copy()

if plot_df.empty:
    raise ValueError("No stocks meet the minimum observation threshold. Lowe
```

```
r min_obs or check unseen_results.")

best = plot_df.nsmallest(top_n, "mae_lstm").sort_values("mae_lstm", ascendin
g=False)
worst = plot_df.nlargest(top_n, "mae_lstm").sort_values("mae_lstm", ascendin
g=True)

cluster_colors = {0: "#1f77b4", 1: "#e377c2"}  # blue / pink
default_color = "#7f7f7f"

fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Best
best_colors = [cluster_colors.get(int(c), default_color) if not np.isnan(c)
else default_color for c in best["cluster"]]
axes[0].barh(best["Symbol"], best["mae_lstm"], color=best_colors)
axes[0].set_title(f"Best {top_n} Stocks for LSTM (Lowest MAE)")
axes[0].set_xlabel("LSTM MAE")
axes[0].set_ylabel("Stock")

# Worst
worst_colors = [cluster_colors.get(int(c), default_color) if not np.isnan(c)
else default_color for c in worst["cluster"]]
axes[1].barh(worst["Symbol"], worst["mae_lstm"], color=worst_colors)
axes[1].set_title(f"Worst {top_n} Stocks for LSTM (Highest MAE)")
axes[1].set_xlabel("LSTM MAE")
axes[1].set_ylabel("Stock")

# Shared legend
handles = [
    plt.Line2D([0], [0], marker='s', color='w', markerfacecolor=cluster_colo
rs[0], markersize=10, label='Cluster 0'),
    plt.Line2D([0], [0], marker='s', color='w', markerfacecolor=cluster_colo
rs[1], markersize=10, label='Cluster 1'),
    plt.Line2D([0], [0], marker='s', color='w', markerfacecolor=default_colo
r, markersize=10, label='Unknown'),
]
fig.legend(handles=handles, loc="upper center", ncol=3, frameon=True)

plt.tight_layout(rect=[0, 0, 1, 0.94])
plt.show()

--------------------------------------------------

# Generate the unseen residual histogram (LSTM)

import matplotlib.pyplot as plt

# Predict on unseen
X_unseen_scaled = (X_unseen - feature_means) / feature_stds
y_pred_unseen = model.predict(X_unseen_scaled, verbose=0).reshape(-1) * y_st
d + y_mean

# Residuals: predicted - actual
residuals = (y_pred_unseen - y_unseen)
```

```
# Plot
plt.figure(figsize=(8, 4))
plt.hist(residuals, bins=50)
plt.title("Residual Distribution on Unseen (Forward Vol)")
plt.xlabel("Residual (Predicted - Actual)")
plt.ylabel("Count")
plt.show()

--------------------------------------------------

plt.figure(figsize=(6, 6))
error = unseen_results["y_pred"] - unseen_results["y_true"]
plt.scatter(unseen_results["y_true"], error, alpha=0.25)
plt.axhline(0, linestyle="--")
plt.xlabel("Actual 21d Vol")
plt.ylabel("Prediction Error")
plt.title("Residuals vs Actual Volatility")
plt.show()

--------------------------------------------------

import matplotlib.pyplot as plt
import seaborn as sns

stocks = ["CTVA", "MAS", "DDOG", "ABNB"]

sns.set_theme(style="darkgrid")
plt.rcParams.update({
    "figure.facecolor": "#0B1020",
    "axes.facecolor": "#0F172A",
    "savefig.facecolor": "#0B1020",
    "text.color": "#E2E8F0",
    "axes.labelcolor": "#E2E8F0",
    "xtick.color": "#E2E8F0",
    "ytick.color": "#E2E8F0",
    "grid.color": "#334155",
})

df = unseen_results.dropna(subset=["Symbol", "y_true", "y_pred"]).copy()
df = df[df["Symbol"].isin(stocks)].copy()

x_col = "Date" if "Date" in df.columns else None

fig, axes = plt.subplots(len(stocks), 1, figsize=(12, 3 * len(stocks)), shar
ex=False)
if len(stocks) == 1:
    axes = [axes]

for ax, sym in zip(axes, stocks):
    g = df[df["Symbol"] == sym].copy()
    if g.empty:
        ax.set_title(f"{sym} (no data)")
        ax.axis("off")
        continue

    if x_col:
```

```
        g = g.sort_values(x_col)
        x = g[x_col]
    else:
        g = g.reset_index(drop=True)
        x = g.index

    ax.plot(x, g["y_true"], color="#22C55E", lw=1.8, label="Actual")  # gree
n
    ax.plot(x, g["y_pred"], color="#F59E0B", lw=1.5, label="LSTM")    # oran
ge
    ax.set_title(sym)
    ax.set_ylabel("21d Volatility")
    ax.legend(loc="upper right", ncol=2, fontsize=8, frameon=False)

fig.suptitle("Actual vs LSTM Predicted Volatility for 4 " \
"Stocks Closest to Cluster Centroids", fontsize=14, y=1.02)
axes[-1].set_xlabel("Date" if x_col else "Observation")

plt.tight_layout()
plt.show()

--------------------------------------------------

# 4 closest stocks to cluster centroids in training set

closest_stocks

--------------------------------------------------

import matplotlib.pyplot as plt
import seaborn as sns

stocks = ["CTVA", "MAS", "DDOG", "ABNB"]

sns.set_theme(style="darkgrid")
plt.rcParams.update({
    "figure.facecolor": "#0B1020",
    "axes.facecolor": "#0F172A",
    "savefig.facecolor": "#0B1020",
    "text.color": "#E2E8F0",
    "axes.labelcolor": "#E2E8F0",
    "xtick.color": "#E2E8F0",
    "ytick.color": "#E2E8F0",
    "grid.color": "#334155",
})

df = unseen_results.dropna(subset=["Symbol", "y_true", "y_pred"]).copy()
df = df[df["Symbol"].isin(stocks)].copy()

x_col = "Date" if "Date" in df.columns else None

fig, axes = plt.subplots(len(stocks), 1, figsize=(12, 3 * len(stocks)), shar
ex=False)
if len(stocks) == 1:
    axes = [axes]
```

```
for ax, sym in zip(axes, stocks):
    g = df[df["Symbol"] == sym].copy()
    if g.empty:
        ax.set_title(f"{sym} (no data)")
        ax.axis("off")
        continue

    if x_col:
        g = g.sort_values(x_col)
        x = g[x_col]
    else:
        g = g.reset_index(drop=True)
        x = g.index

    ax.plot(x, g["y_true"], color="#22C55E", lw=1.8, label="Actual")  # gree
n
    ax.plot(x, g["y_pred"], color="#F59E0B", lw=1.5, label="LSTM")    # oran
ge
    ax.set_title(sym)
    ax.set_ylabel("21d Volatility")
    ax.legend(loc="upper right", ncol=2, fontsize=8, frameon=False)

fig.suptitle("Actual vs LSTM Predicted Volatility for 4 " \
"Stocks Closest to Cluster Centroids", fontsize=14, y=1.02)
axes[-1].set_xlabel("Date" if x_col else "Observation")

plt.tight_layout()
plt.show()

--------------------------------------------------

# ===== CODE APPENDIX =====

from IPython.display import Markdown

code_cells = []

for cell in get_ipython().user_ns['_ih']:
    if cell.strip():
        code_cells.append(cell)

print("===== CODE APPENDIX =====\n")
for c in code_cells:
    print(c)
    print("\n" + "-"*50 + "\n")

--------------------------------------------------
```